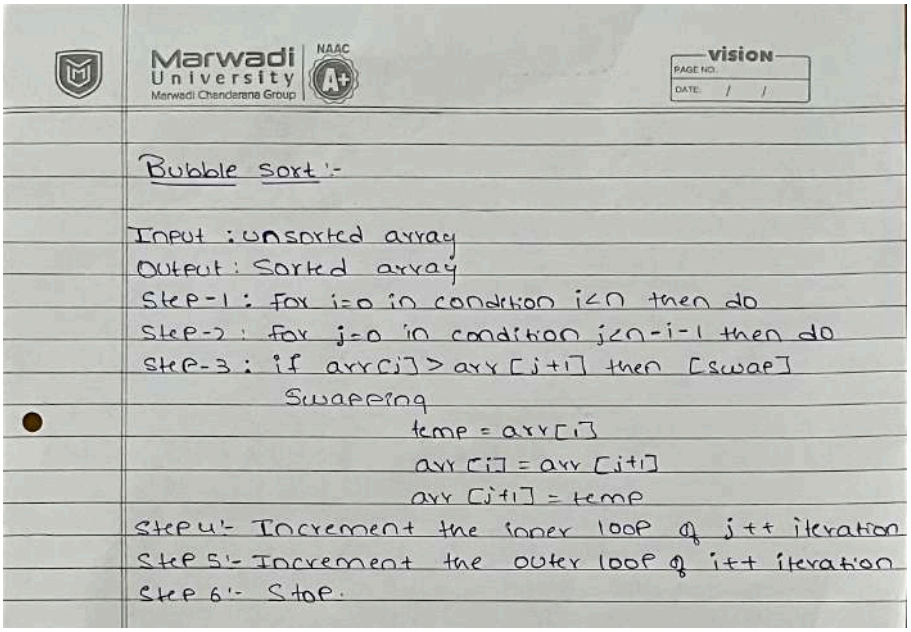
PRACTICAL - 1

AIM : Implementation and Time analysis of sorting algorithms, Bubble sort , Selection sort , Insertion sort , Merge sort and Quick sort.

ALGORITHM : // Bubble sort



PROGRAM :

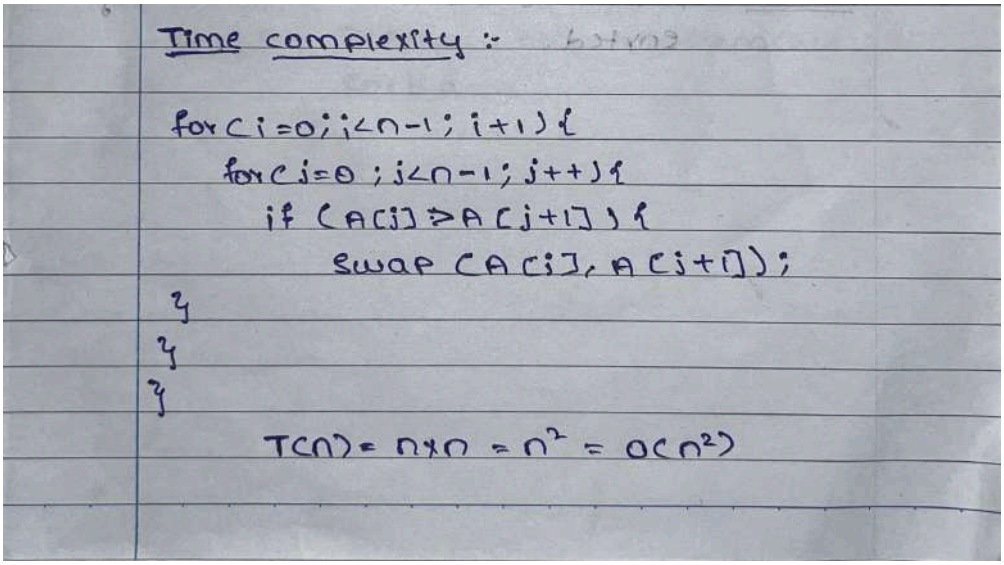
```
#include <iostream>
using namespace std;

int main()
{
   int n;

   cout << "Enter the number of elements: ";
   cin >> n;

   int arr[n];

   // Enter elements on the same line
   cout << "Enter " << n << " elements: ";

   for (int i = 0; i < n; i++)

{
```

```
cin >> arr[i];
    }

    // Bubble Sort
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            // Compare adjacent elements
            if (arr[j] > arr[j + 1])
            {
                // Swap
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }

    // Display sorted array
    cout << "Sorted array: ";

    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }

    return 0;
}
```

OUTPUT :

```
SYNC STORAGE/UNIVERSITY STUFF/DAA_LAB on  master
❯ ./BubbleSort
Enter the number of elements: 4
Enter 4 elements: 21 09 06 07
Sorted array: 6 7 9 21 %
```

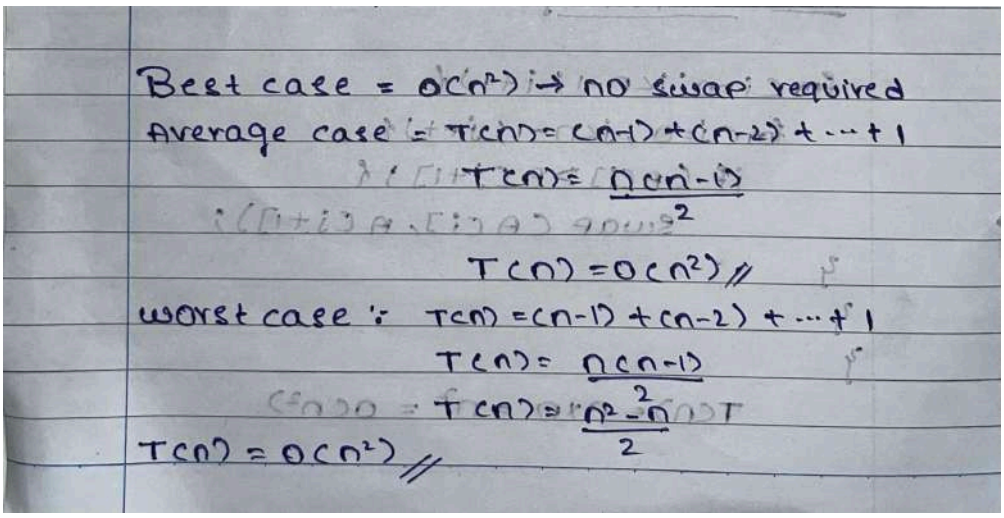
TIME COMPLEXITY :



Time complexity :-

```
for(i=0;i<n-1;i+1){
   for(j=0;j<n-1;j++){
      if(A[j]>A[j+1]){
         swap(A[j],A[j+1]);
   }
}
}
```

$T(n) = n \times n = n^2 = O(n^2)$



Best case = $O(n^2)$ → no swap required

Average case :- $T(n) = (n-1) + (n-2) + \cdots + 1$

$T(n) = \frac{n(n-1)}{2}$

$T(n) = O(n^2)$ //

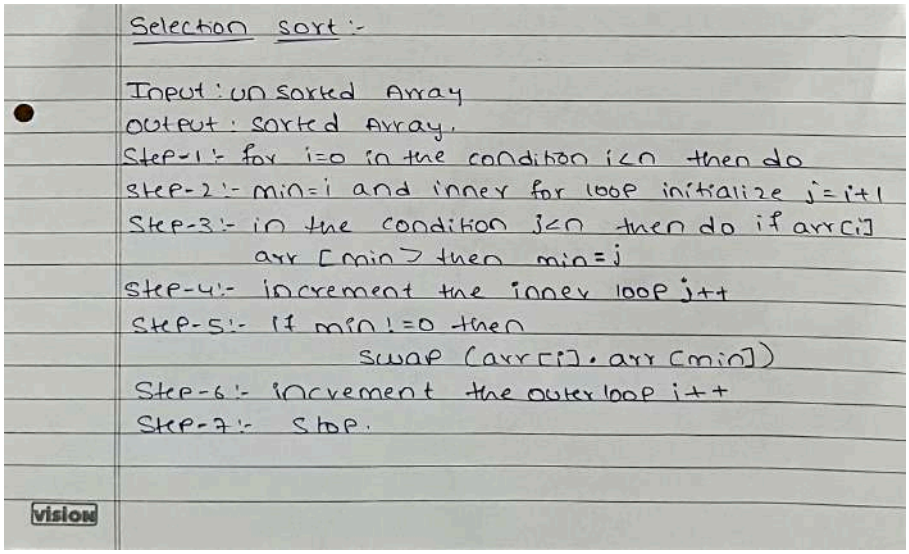
worst case :- $T(n) = (n-1) + (n-2) + \cdots + 1$

$T(n) = \frac{n(n-1)}{2}$

$T(n) = \frac{n^2 - n}{2}$

$T(n) = O(n^2)$ //

ALGORITHM : // Selection sort

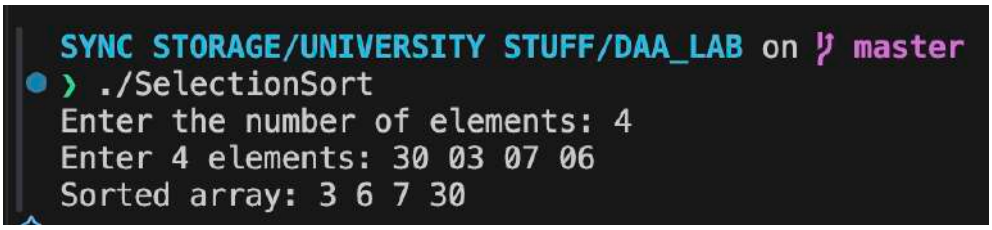


PROGRAM :

```
#include <iostream>
using namespace std;

// Function to perform Selection Sort
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;

        // Find the smallest element in the unsorted part
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }

        // Swap the smallest element with the current element
        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;

    }
}

int main() {
    int n;
```

```
cout << "Enter the number of elements: ";
    cin >> n;

    int arr[n];

    cout << "Enter " << n << " elements:\n";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    selectionSort(arr, n);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    cout << endl;

    return 0;
}
```

OUTPUT :



TIME COMPLEXITY :

Time complexity :-

```
for (int i=0 ; i<n-1 ; i++) {
    int minIndex = i ;
  for (int j= i+1 ; j<n ; j++) {
    if (A[j] < A [minIndex ]){
  }
  }
    swap (A[i], A[min Index]) ;
}
}
```

DOMS Page No.
Date / /

Best case :-

$T(n) = (n-1) + (n-2) + (n-3) + \cdots + 1$

$T(n) = \frac{n(n-1)}{2}$

$T(n) = \frac{n^2 - n}{2}$

$T(n) = O(n^2)$

$i=0 \rightarrow n-1$
$i=1 \rightarrow n-2$
$i=2 \rightarrow n-3$
$i = n-2 \rightarrow 1$ comparison

Average case :-

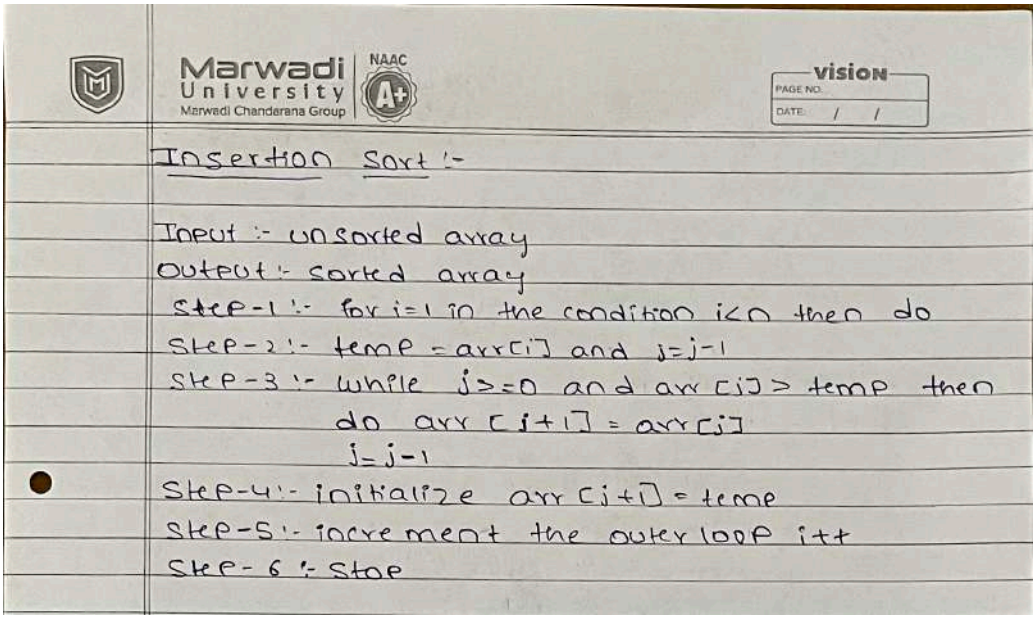
$T(n) = 1+2+3+\cdots+(n-1)$

$T(n) = \frac{n(n-1)}{2}$

$T(n) = O(n^2)$

worst case :- $T(n) = O(n^2)$

ALGORITHM : // Insertion sort

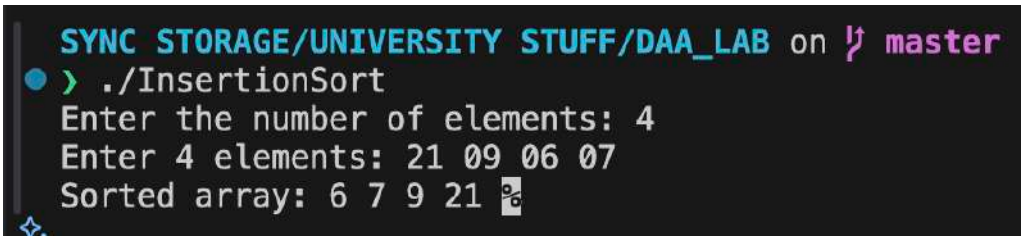


PROGRAM :

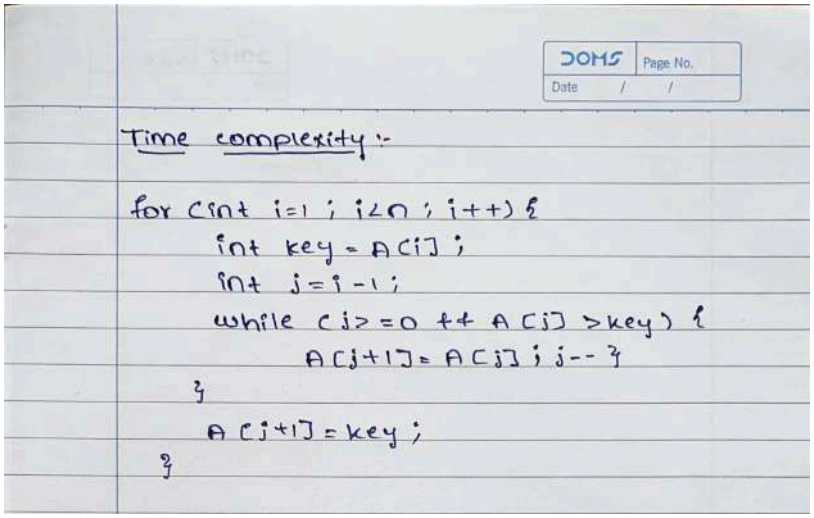
```
#include <iostream>
using namespace std;

int main()
{
   int n;

   cout << "Enter the number of elements: ";
   cin >> n;

   int arr[n];

   // Enter elements on the same line
   cout << "Enter " << n << " elements: ";

   for (int i = 0; i < n; i++)
   {


     cin >> arr[i];
   }

   // Insertion Sort
   for (int i = 1; i < n; i++)
```

```
{
    int key = arr[i];
    int j = i - 1;

    // Move elements greater than key one position ahead
    while (j >= 0 && arr[j] > key)
    {
        arr[j + 1] = arr[j];
        j--;
    }

    // Insert key at the correct position
    arr[j + 1] = key;
  }

  // Display sorted array
  cout << "Sorted array: ";

  for (int i = 0; i < n; i++)
  {
    cout << arr[i] << " ";
  }

  return 0;
}
```

OUTPUT :

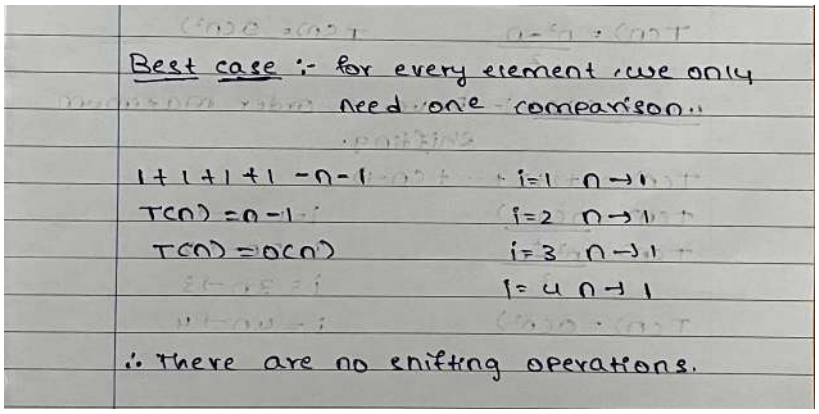


TIME COMPLEXITY :



Time complexity :-

```
for (int i=1 ; i<n ; i++) {
    int key = A[i] ;
    int j = i-1 ;
    while (j>=0 && A[j] >key) {
        A[j+1] = A[j] ; j-- }
  }
  A[j+1] = key ;
}
```

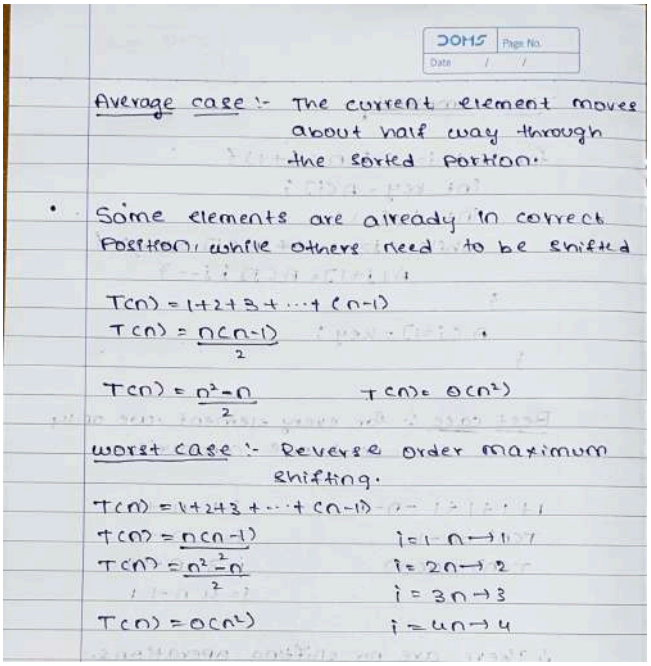


Best case :- for every element, we only need one comparison.

1+1+1+1 ... n-1

T(n) = n-1 i=1 n→1
T(n) = O(n) i=2 n→1
i=3 n→1
i=4 n→1

∴ There are no shifting operations.



Average case :- The current element moves about half way through the sorted portion.

- Some elements are already in correct position, while others need to be shifted

$T(n) = 1+2+3+\cdots+(n-1)$

$T(n) = \frac{n(n-1)}{2}$

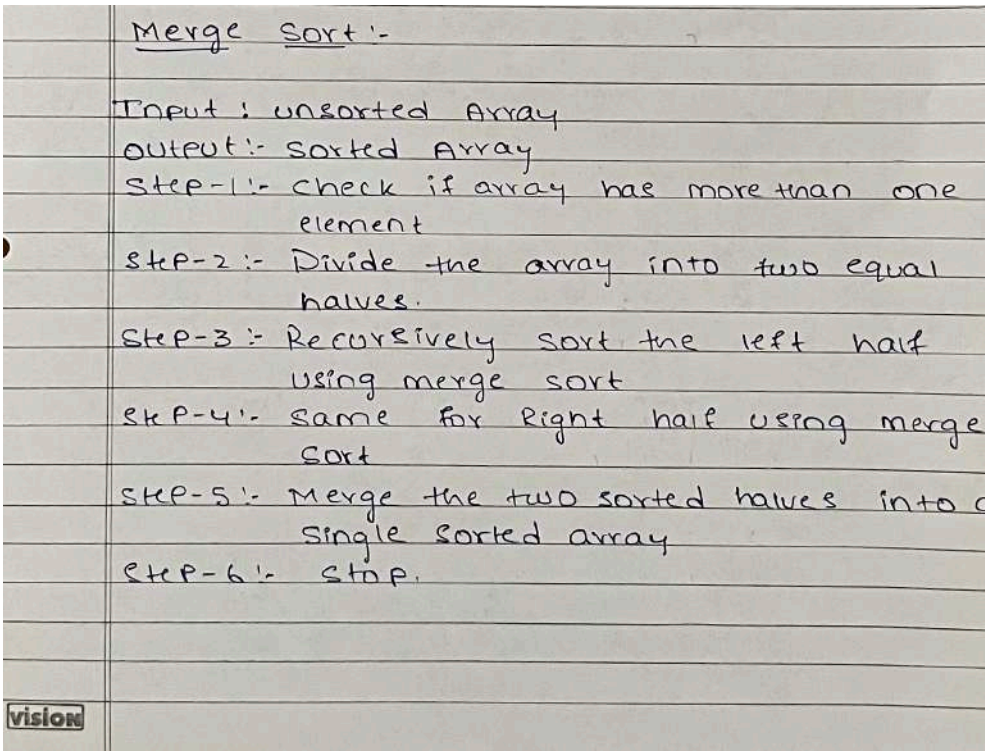
$T(n) = \frac{n^2-n}{2}$ $T(n) = O(n^2)$

Worst case :- Reverse order maximum shifting.

$T(n) = 1+2+3+\cdots+(n-1)$

$T(n) = \frac{n(n-1)}{2}$ i=1 n→1
$T(n) = \frac{n^2-n}{2}$ i=2n→2
i=3n→3
$T(n) = O(n^2)$ i=4n→4

ALGORITHM : // Merge sort


Merge Sort :-

Input : unsorted Array
output :- sorted Array
step-1 :- check if array has more than one element
step-2 :- Divide the array into two equal halves.
step-3 :- Recursively sort the left half using merge sort
step-4 :- same for right half using merge sort
step-5 :- Merge the two sorted halves into a single sorted array
step-6 :- stop.

vision

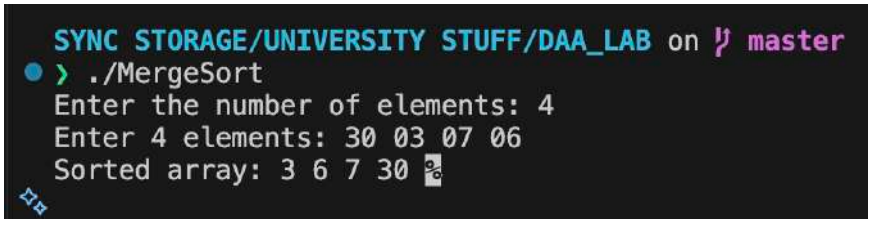
PROGRAM :

```
#include <iostream>
using namespace std;

// Merge two sorted parts
void merge(int arr[], int low, int mid, int high)
{
   int i = low;
   int j = mid + 1;
   int k = 0;

   int temp[100];

   // Compare elements from both parts
   while (i <= mid && j <= high)
   {
      if (arr[i] < arr[j])
      {
         temp[k] = arr[i];
         i++;
      }
```

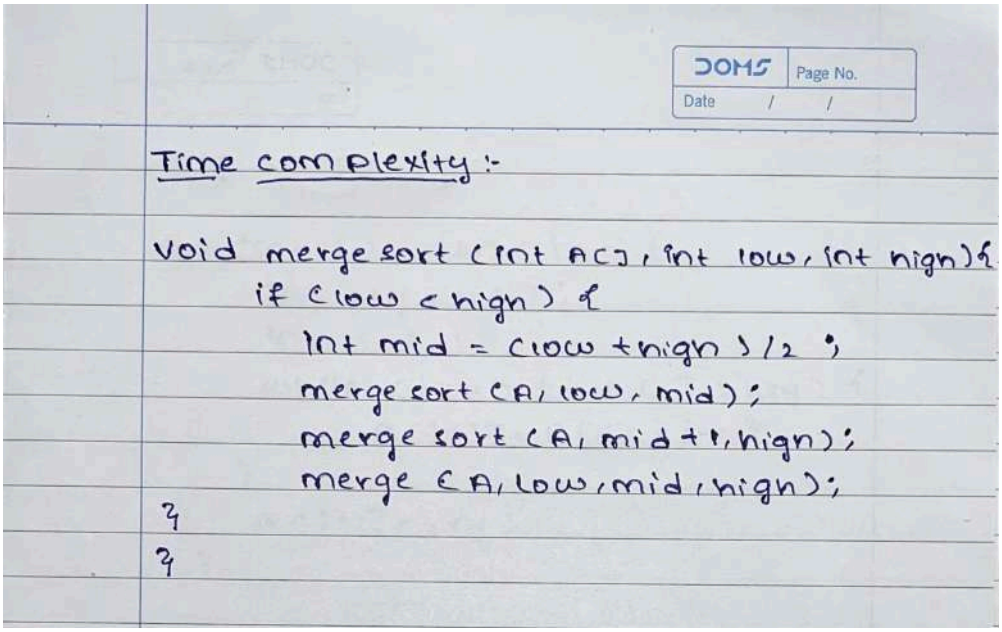
```
else
        {
           temp[k] = arr[j];
           j++;
        }

        k++;
    }

    // Copy remaining elements from left part
    while (i <= mid)
    {
        temp[k] = arr[i];
        i++;
        k++;
    }

    // Copy remaining elements from right part
    while (j <= high)
    {
        temp[k] = arr[j];
        j++;
        k++;
    }

    // Copy sorted elements back to original array
    for (i = low, k = 0; i <= high; i++, k++)
    {
        arr[i] = temp[k];
    }
}

// Merge Sort
void mergeSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int mid = (low + high) / 2;

        // Divide the array
        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);
```

```
        // Merge the sorted parts
        merge(arr, low, mid, high);
    }
}

int main()
{
    int n;

    cout << "Enter the number of elements: ";
    cin >> n;

    int arr[n];

    // Enter elements on the same line
    cout << "Enter " << n << " elements: ";

    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }

    // Call Merge Sort
    mergeSort(arr, 0, n - 1);

    // Display sorted array
    cout << "Sorted array: ";

    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }

    return 0;
}
```

OUTPUT :

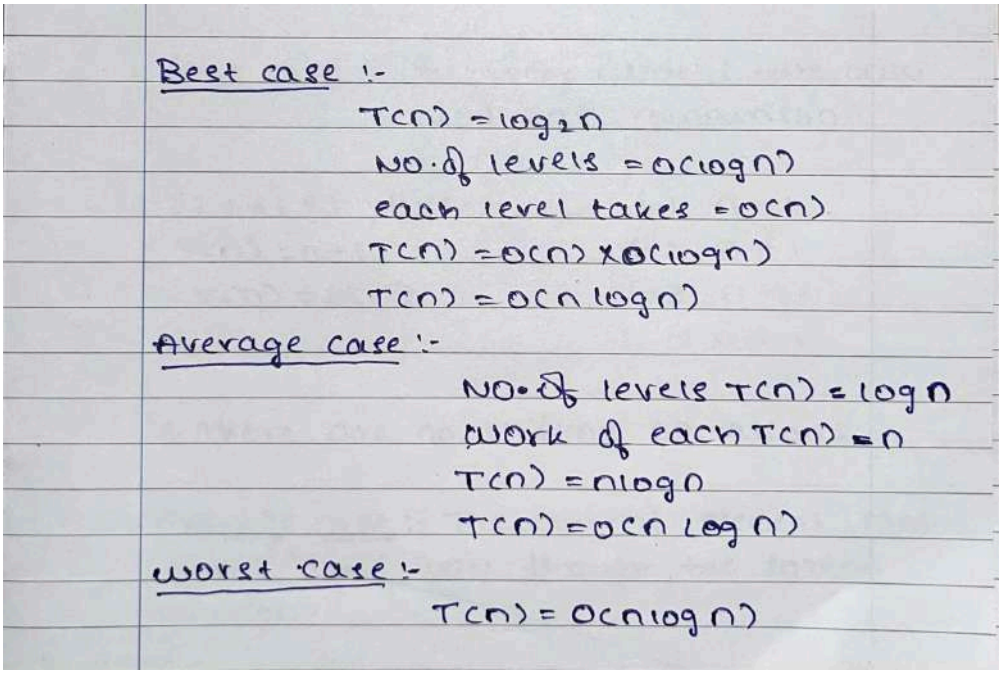


TIME COMPLEXITY :



Time complexity :-

```
void mergesort (int A[], int low, int high){
    if (low < high) {
        int mid = (low + high) / 2 ;
        mergesort (A, low, mid);
        mergesort (A, mid+1, high);
        merge (A, low, mid, high);
    }
}
```



Best case :-

$T(n) = \log_2 n$

No. of levels $= O(\log n)$

each level takes $= O(n)$

$T(n) = O(n) \times O(\log n)$

$T(n) = O(n \log n)$

Average case :-

No. of levels $T(n) = \log n$

work of each $T(n) = n$

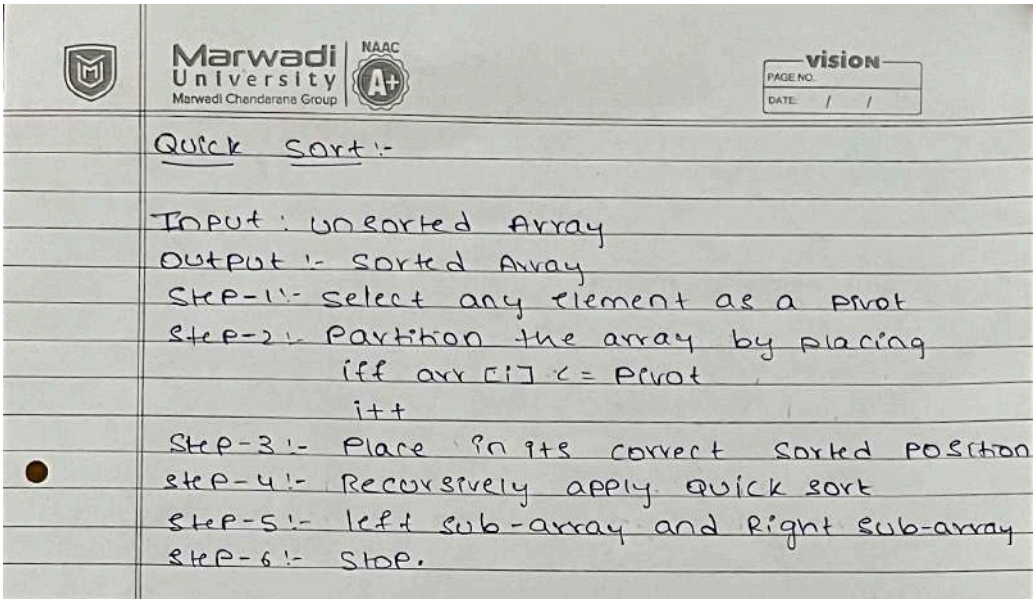
$T(n) = n \log n$

$T(n) = O(n \log n)$

worst case :-

$T(n) = O(n \log n)$

ALGORITHM : // Quick sort



PROGRAM :

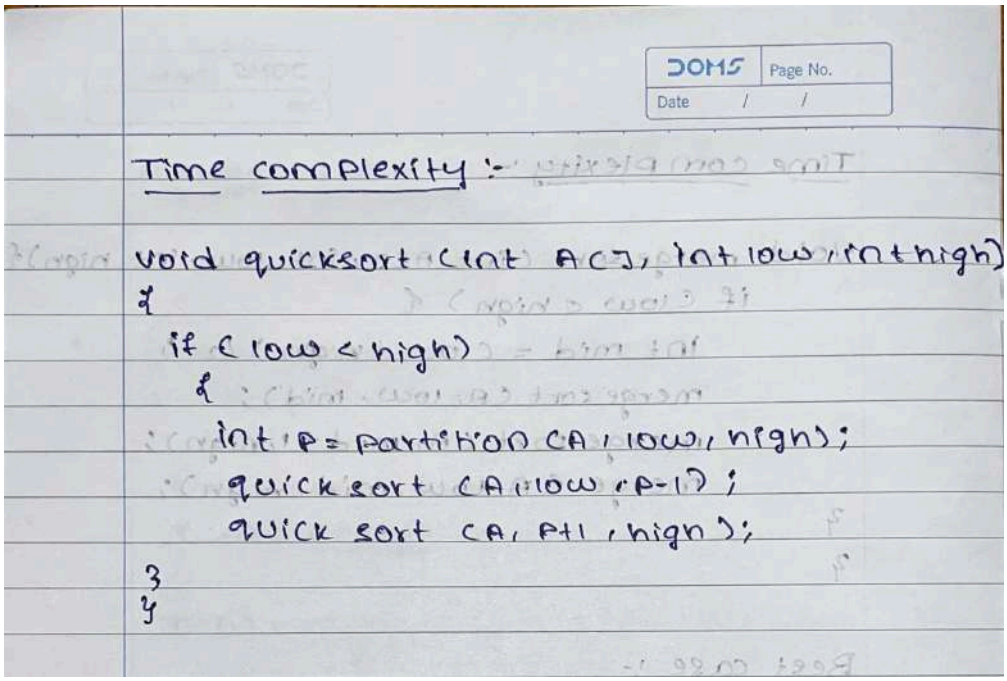
```
#include <iostream>
using namespace std;

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }

    swap(arr[i + 1], arr[high]);
    return i + 1;
}
void quickSort(int arr[], int low, int high) {


 if (low < high) {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
```

```
  quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n;

    cout << "Enter the number of elements: ";
    cin >> n;

    int arr[n];

    cout << "Enter the elements:\n";
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    quickSort(arr, 0, n - 1);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    cout << endl;
    return 0;
}
```

OUTPUT :

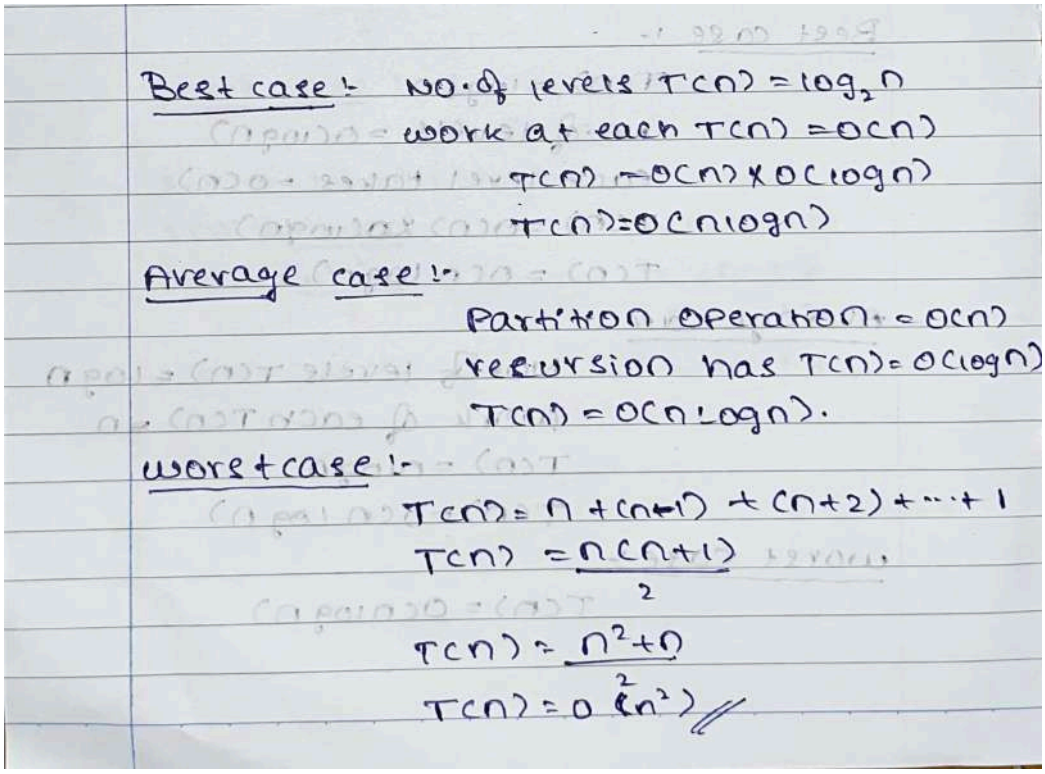
```
SYNC STORAGE/UNIVERSITY STUFF/DAA_LAB on  master
❯ ./QuickSort
Enter the number of elements: 4
Enter the elements: 09 11 07 30
Sorted array: 7 9 11 30
```

TIME COMPLEXITY :



Time complexity :-

```
void quicksort (int A[], int low, int high)
{
  if (low < high)
    {
     int p = partition (A, low, high);
     quicksort (A, low, p-1);
     quick sort (A, p+1, high);
}
}
```



Best case :- No. of levels $T(n) = \log_2 n$

work at each $T(n) = O(n)$

$T(n) = O(n) \times O(\log n)$

$T(n) = O(n \log n)$

Average case :-

Partition operation $= O(n)$

recursion has $T(n) = O(\log n)$

$T(n) = O(n \log n)$.

worst case :-

$T(n) = n + (n-1) + (n-2) + \cdots + 1$

$T(n) = \frac{n(n+1)}{2}$

$T(n) = \frac{n^2+n}{2}$

$T(n) = O(n^2)$

CONCLUSION : I successfully implemented and studied different sorting algorithms such as Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort. I understood how each algorithm arranges the elements of an array in ascending or descending order. I also performed the time complexity analysis of these algorithms and understood that their efficiency varies depending on the approach used. Through this practical, I learned how to choose a suitable sorting algorithm based on the size of the data and the required performance.